

Auditing Actual Execution and Output Portability in Quantized LLM Inference: A Measurement Study of llama.cpp

Anonymous

October 2026 (technical note)

Abstract

We measured whether the same quantized GGUF model, the same prompt, and fully deterministic greedy decoding (temperature 0) produce the same output across the compute backends of llama.cpp (release b11327, official binaries, SHA256-recorded) on a 12-machine multi-vendor fleet (NVIDIA, AMD, Intel; Linux and Windows). Auditing *which device actually executed each run* — via per-prompt wall-time ratios against the same-box CPU build — turned out to be the decisive methodological step: three distinct failure classes silently served inference from the CPU while presenting as GPU backends, including the official ROCm build on four AMD hosts (its backend library requires ROCm userspace SONAMEs one major version newer than stock installs, and no Linux runtime is bundled), a CUDA-13 build deprived of its runtime libraries by our own launch environment, and a virtualized host with no GPU passthrough at all. On runs verified to execute on the GPU, byte-exact agreement with the same-box CPU reference is quantization-dominated and monotone — F16 88.0%, Q8_0 35.3%, Q4_K_M 18.0% of 8,670 prompt comparisons — and nearly identical rates appear between *different builds’ CPU implementations* on the same host (F16 92.7%, Q8_0 40.5%, Q4_K_M 17.7%), and even between the same CPU build on different hosts. Divergence is concentrated at near-ties: at all 7,875 first divergences, the alternative token lies within the reference’s recorded top-5 in 99.89% of events (median logprob gap 0.062 nats, p90 0.231). Identical stacks are perfectly deterministic: 100% agreement across two different physical machines with identical GPU/driver/binary, and 100% across a same-machine 12-hour re-run of 51 cells. We release the harness, per-token logprob records, and the device-truth audit method. Practical conclusions: verify actual execution placement before labeling any measurement with a backend name; treat quantized artifacts as non-portable across stacks at Q4-class precision; and do not use default-tool perplexity as a conformance signal.

1 Introduction

A greedy-decoded output is usually treated as a property of the model and the prompt. This note reports a measurement study of the other variable: the numerical stack that executes the model. We ask two questions. First, *does the intended backend actually execute the workload?* Second, *when the stack differs, how much does greedy output differ, and where?*

The first question became the pivot of the study. We began by comparing backends on a multi-vendor fleet and initially read the results as backend behavior. A device-truth audit — comparing every run’s per-prompt generation wall time against the same-box CPU build — revealed that three of our “GPU backend” columns had silently served from the CPU: the official ROCm build on all four AMD hosts, the CUDA-13 build on both hosts offering it, and CUDA on a virtualized

host. Only the first is an upstream packaging gap; the second was a launch-environment mistake in our own harness (which had passed the runtime-library path to device enumeration but not to the server process); the third was a virtual machine with no GPU passthrough. All three served HTTP requests normally, with no error on the console, and two of the three even enumerated a device under `-list-devices` — enumeration, we conclude, is not evidence of execution.

The audit changed the study’s claims in both directions. It killed findings we had briefly believed (a “CUDA-toolkit changes everything” comparison; a “perplexity defaults differ between tools” hypothesis) because those comparisons had in fact measured CPU implementations against each other. It also sharpened the findings that survive: the conformance structure of runs *verified* to differ in execution, the near-tie anatomy of divergence, and the perfect determinism of identical stacks.

2 Method

Fleet. 12 physical machines (13 run labels; one machine was measured twice, 12 hours apart, and its re-run is used only as a stability control): NVIDIA (RTX 5060 Ti, RTX PRO 6000 Blackwell, TITAN RTX, RTX 3060 Ti/Windows), AMD (RX 7900 XTX $\times 2$, Radeon AI PRO R9700, Strix Halo 8060S), Intel (Arc A770 Linux + Windows, Arc B60); hosts span several CPU generations and both desktop OSes.

Engine and models. llama.cpp release b11327, official prebuilt binaries only, SHA256 recorded. 17 core GGUF models: Qwen2.5-Instruct 0.5B/1.5B/3B and SmolLM2-Instruct 135M/360M/1.7B at Q4_K_M, Q8_0, and F16/FP16 (official Qwen-org quantizations; community re-quantizations for SmolLM2 — provenance is a caveat for family comparisons).

Protocol. 34 fixed raw-completion prompts (no chat template; four categories), greedy decoding (temperature 0), 128-token cap, prompt caching disabled, one request at a time. The server protocol records, for every generated token on *every* backend, the chosen id, its logprob, and the top-5 ids with logprobs. Perplexity is measured by the same release’s `llama-perplexity` on a fixed ~ 5 k-token corpus. The same-box CPU build is the reference for all per-cell comparisons.

Device-truth audit. For every (machine, backend) cell we compute the median over models of the ratio of median per-prompt generation wall time to the same-box CPU build’s. Observed ratios bifurcate cleanly (≈ 1.0 vs ≤ 0.67); we verified a sample of cells at the ≈ 1.0 end with process maps, VRAM readings, and library-load checks (§3). Cells at ≈ 1.0 are classified *CPU-fallback* and analyzed as a separate population, not as GPU backends.

3 Finding 1: silent CPU fallback

Three failure classes, one user-visible behavior: the server accepts requests and generates normally; nothing on the console distinguishes it from a GPU deployment.

(a) **Missing backend userspace (upstream-relevant).** The official ROCm build’s `libggml-hip.so` requires `libamdhip64.so.7`, `libhipblas.so.3`, and `librocblas.so.5`. Stock Ubuntu hosts with ROCm 6.3.0 provide `.so.6/.so.2/.so.4`; the release bundles no Linux ROCm runtime (a Windows runtime zip exists; the Linux equivalent does not). The backend library therefore never loads on any of our four AMD hosts — including an RX 7900 XTX, an officially supported target — `-list-devices` prints `(none)`, and the server runs fully on the CPU, *including with an explicit `-ngl 999`*: process maps show no HIP library, VRAM stays idle, and generation proceeds at exactly CPU-build speed. The perplexity tool is subject to the same backend failure, so its values are host-CPU values; on our data the “ROCm” perplexity matched the CPU build’s in 68 of 68 cells, and every “ROCm” generation was bit-identical to the CPU build’s on the same host. Related reports

exist upstream for Windows and other setups; our contribution is the Linux SONAME detail, the cross-host confirmation, and the benchmarking consequence: quality measurements taken this way silently measure the CPU.

(b) Runtime-library environment mistakes (our own). Our harness passed the CUDA runtime-library path (`LD_LIBRARY_PATH`) to device enumeration and to the perplexity tool, but not to the server process. On hosts where `libcudart.so.13` exists only in the release’s runtime tarball, the CUDA-13 backend loaded for `-list-devices` — which therefore reported the GPU — but not for the server, which silently served from the CPU. An on-box A/B check isolates the effect: with the environment set, the CUDA-13 build offloads under the default `-ngl auto` at 667 tok/s; without it, even `-ngl 999` runs on the CPU at 203 tok/s with 2 MiB VRAM in use. The CUDA-12.8 build was unaffected only because those hosts carry system CUDA 12.9 libraries. We retract the preliminary “CUDA-13 behaves like a different backend” observation that this artifact produced, and note the general lesson: `-list-devices` output reflects the environment of whatever process ran it, which is not necessarily the environment of the server.

(c) Absent hardware. One host, labeled as a virtualized RTX 4090, has no NVIDIA device on its guest PCI bus at all (no `/dev/nvidia*`, no driver library, only an Intel iGPU on `lspci`). Every backend on it was CPU execution by construction. We retain this host’s cells only within the CPU-fallback population and note that our earlier “virtualized 4090” hardware label was wrong.

Detection. All three classes were invisible to `-list-devices` in at least one instance, and none produced a startup error. The wall-time-ratio audit caught all three, and it is cheap: it requires only a CPU-build baseline on the same host. We recommend it as a standing check in any benchmarking or conformance setup, alongside post-launch verification that the backend library is actually mapped in the server process.

4 Finding 2: conformance on verified runs

On runs verified to execute on the GPU (CUDA on 4 machines, Vulkan on 10; 255 cells after removing the re-run), byte-exact agreement with the same-box CPU reference is:

Population	Q4_K_M	Q8_0	F16/FP16
GPU vs same-box CPU (n: 3060/3060/2550)	18.0%	35.3%	88.0%
Other builds’ CPU impl. vs same-box CPU (n: 1428/1428/1190)	17.7%	40.5%	92.7%
Same CPU build, different host (per pair)	19.6–20.1%	39.2–40.7%	92.9–94.7%

Three observations. First, quantization dominates: within every population the rates are monotone in precision, and the pooled “~45–48% agreement” that a fixed quant mix produces is a mixture artifact — per-precision rates span 18% to 93%. Second, the comparison type is a second-order effect: GPU-vs-CPU, one build’s CPU implementation vs another’s (including the ROCm and CUDA-13 fallback columns, reclassified), and even the same CPU build on different host CPUs all land within a few points of each other at each precision. Byte-exact portability across *any* numerical change of stack is essentially gone at Q4-class precision and partially preserved at F16. Third, the CPU reference is itself stack-relative: different hosts running the same CPU build agree with each other no more than GPU backends agree with their local CPU, and CPU outputs form bit-identical equivalence classes across subsets of our hosts. “Same result as CPU” therefore means “same result as this host’s CPU build”.

5 Finding 3: identical stacks are deterministic

Two controls bound the divergence findings from above. Two physically distinct machines with the same GPU (RX 7900 XTX), same driver, same OS, and same Vulkan binary produced bit-identical generations for 578/578 prompt comparisons — cross-machine determinism is perfect when the stack is identical. The same machine re-run 12 hours later (51 cells, 1,734 prompts; CUDA, CUDA-13, and Vulkan) reproduced its earlier outputs bit-for-bit in 100.0% of comparisons. Divergence in §4 is therefore a property of differing stacks, not of nondeterminism within a stack. (We separately note that the engine’s own documentation warns that prompt-cache reuse can introduce run-to-run nondeterminism via batch-shape-dependent prefill/decode arithmetic; our protocol runs with caching disabled.)

6 Finding 4: divergence anatomy

At every first-divergence position (7,875 events; 6,903 after removing identical-stack duplicates), the diverging backend’s chosen token lies within the reference’s recorded top-5 in 99.89% of events (9 exceptions, all in one SmolLM2 Q4_K_M cell family). Because both sides’ top-5 logprobs are recorded, the gap can be measured in both distributions: the reference’s preference for its own argmax over the backend’s choice has median 0.062 nats (p90 0.231, p99 0.564); the backend’s preference for its choice over the reference’s has median 0.059 nats (p90 0.226). Hand inspection of sampled divergences shows rank-1 $\leftrightarrow$ 2 swaps. Divergence at greedy decoding is therefore concentrated at near-ties — compatible with the rounding-order mechanism assumed in prior work, though our data does not by itself exclude small systematic implementation differences that would also preferentially flip low-margin decisions.

7 Related work

Cross-platform numerical deviation in DNN inference, with platform equivalence classes, was established for vision models by Schlögl et al. (NeurIPS 2023). He (Thinking Machines Lab, 2025) identifies batch-shape dependence as the root cause of temperature-0 nondeterminism and ships batch-invariant kernels; the engine’s own cache-reuse warning is an instance of this class. Yuan et al. (2025, v2: “Understanding and Mitigating Numerical Sources of Nondeterminism in LLM Inference”) measure greedy divergence across GPU type, count, batch size and precision at bf16 scale and propose LayerCast; Du et al. (TMLR 2026) measure cross-precision greedy divergence with margin-thresholded mitigation; Dutta et al. (NeurIPS 2024) show aggregate accuracy metrics hide flips in compressed models — our perplexity observations are an instance. Pape et al. (2026) quantify backend-choice effects across engines including llama.cpp. Deterministic or verified inference is addressed by Gond et al. (LLM-42, 2026), Cooper et al. (2026, fixed-reduction GEMMs), and TOPLOC (Ong et al., 2025) for verifiable inference. In software engineering, CRADLE (ICSE 2019) cross-validated deep-learning library backends; our harness follows that tradition for autoregressive LLM inference, and its device-truth audit extends it with an execution-placement check. Training-side tooling variance is covered by Zhuang et al. (2021) and Pham et al. (ASE 2020).

8 Limitations

One engine, one release (b11327) — rates will drift with any release, while the silent-fallback classes and the audit method are the durable content. Models are small (135M–3B) by design (exact byte-

level comparison needs exactly reproducible runs; larger models are future work). Raw-completion prompts without chat templates raise continuation entropy relative to deployed chat use. The near-tie evidence is compatible with, but does not uniquely establish, rounding-order causation. Perplexity comparisons across backends are confounded by the fallback classes and by prefill-vs-decode kernel differences. SYCL contributed no complete cells (Linux binaries failed to load the oneAPI runtime; the Windows build loaded, demonstrably offloaded, then crashed mid-run).

9 Conclusion

For practitioners: (1) verify what executed — the wall-time ratio against a CPU baseline is a cheap, reliable detector of silent CPU fallback, and `-list-devices` is not; (2) expect byte-exact greedy output to be non-portable across any stack change at Q4-class precision (18% agreement), partially portable at Q8 (35%), and mostly preserved at F16 (88%); (3) identical stacks are perfectly deterministic — pin backend, binary, GPU, and driver if you need reproducibility; (4) do not treat default-tool perplexity as evidence of behavioral equivalence. We release the harness, the per-token logprob dataset, and the audit tooling.